

Ruby profiling

Stackprof

Stackprof is a sampling CPU profiler for Ruby. Due to its low overhead, it can be used to profile in production.

This may be particularly useful when diagnosing a process that is stuck or is exhibiting unexpectedly high CPU usage.

The configuration and usage of Stackprof is documented in the GitLab performance docs.

What follows are instructions for how to profile specific targets.

puma (api, git, web)

From a puma host, e.g. `api-cny-01-sv-gprd.c.gitlab-production.internal`:

Initiate 30 second CPU profile across all puma workers:

```
sudo pkill -USR2 -f puma:
```

Alternatively, a single worker can be targeted:

```
kill -USR2 $PID
```

After 30 seconds, check for profiles to show up, one file per process:

```
ls -lah /tmp/stackprof.*
```

```
-rw-r--r-- 1 git git 767K Jul 15 12:06 /tmp/stackprof.25246.83c4ef378bef.profile
...
```

To inspect the profile:

```
sudo /opt/gitlab/embedded/bin/chpst -e /opt/gitlab/etc/gitlab-rails/env -u git:git -U git:git
```

To get a per-line breakdown of a particular method:

```
sudo /opt/gitlab/embedded/bin/chpst -e /opt/gitlab/etc/gitlab-rails/env -u git:git -U git:git
```

Now, those profiles can be aggregated up into a single “stackcollapse” file:

```
find /tmp -maxdepth 1 -name 'stackprof.*' -mmin -5 | xargs -n1 sudo /opt/gitlab/embedded/bin
```

That stackcollapse file can then be copied off of the host and run through flamegraph to produce a flamegraph visualization:

```
cat stacks.api-cny-01-sv-gprd.gz | gunzip | flamegraph.pl > flamegraph.svg
```

sidekiq on kubernetes

To initiate profiling for all sidekiq processes on a pod in kubernetes (from a console host such as `console-01-sv-gprd.c.gitlab-production.internal`).

First find the pod name:

```
kubectl get pods -n gitlab
```

Now you can initiate the profile collection:

```
kubectl exec -n gitlab -it gitlab-sidekiq-urgent-other-v1-8bc47b7b4-d6g8f -- /usr/bin/pkill
```

List profiles:

```
kubectl exec -n gitlab -it gitlab-sidekiq-urgent-other-v1-8bc47b7b4-d6g8f -- /bin/ls -lah /t
```

Peek at the profile:

```
kubectl exec -n gitlab -it gitlab-sidekiq-urgent-other-v1-8bc47b7b4-d6g8f -- /srv/gitlab/bin
```

And generate a stackcollapse file:

```
kubectl exec -n gitlab -it gitlab-sidekiq-urgent-other-v1-8bc47b7b4-d6g8f -- /srv/gitlab/bin
```

For more interactive diagnosis, you can also attach to the pod:

```
kubectl exec -n gitlab -it gitlab-sidekiq-urgent-other-v1-8bc47b7b4-d6g8f /bin/bash
```

rbspy

rbspy is a less invasive profiler that runs outside of the Ruby process.

One thing worth noting is that it profiles all stacks, not just on-CPU ones. It is more effective at diagnosing wall clock time spent by threads than time spent on CPU. For CPU profiling, you'll want to use stackprof.

It is available on all rails and gitaly hosts.

To profile all puma processes for 30 seconds (the default sample rate is 100hz):

```
sudo rbspy record -p $(pgrep -n -f 'puma ') --subprocesses --duration 30
```

To profile a single puma worker:

```
sudo rbspy record -p $(pgrep -n -f 'puma:') --duration 30
```

This will write a flamegraph to `~/.cache/rbspy`. You can then `scp` it to your local machine for your viewing pleasure.

jeprof

Since we are using the `jemalloc` allocator, we can use its profiling capabilities. This can be done by configuring the `MALLOC_CONF` environment variable.

This profiler is a sampling profiler that grabs a stack every 512 allocated kilobytes. It also tracks when those sampled allocations are freed. Over time, that can give an idea of where long-lived allocations are originating from.

A sample configuration:

```
MALLOC_CONF='prof:true,prof_prefix:/tmp/jeprof.out,lg_prof_interval:30'
```

This will enable profiling, write dumps to /tmp, and write out one dump every 2^{30} bytes = ~every 1GB allocated.

Other modes such as `prof_gdump` are also supported, see the `jemalloc(3)` man page for more details.

To process the profile, you can use `jeprof`:

```
/opt/gitlab/embedded/bin/jeprof /opt/gitlab/embedded/bin/ruby /tmp/jeprof.out.9854.44.i44.h
```

This can be used to get top allocation sites, dump out callgrind or svg profiles.

In order to get a flamegraph, you can apply this patch to `jeprof`.

Flamegraphs can then be generated via:

```
jeprof /opt/gitlab/embedded/bin/ruby /tmp/jeprof.out.9854.44.i44.heap --collapse | flamegrap
```

And for a reverse flamegraph:

```
jeprof /opt/gitlab/embedded/bin/ruby /tmp/jeprof.out.9854.44.i44.heap --collapse | flamegrap
```